FloodLite: A Multi-Platform Edge-Deployment Benchmark for Lightweight Flood Segmentation

Tasfia Noor Chawdhury[a], Md Ibrahim Khalil[b,*], Md Ikramul Haque[c]

[a] *[Affiliation for T. N. Chawdhury — to be confirmed],*
[b] *Sensa AS, Oslo, Norway*
[c] *[Affiliation for M. I. Haque — to be confirmed],*

Abstract

Operational flood-extent mapping is bottlenecked not by segmentation accuracy on academic benchmarks but by the gap between published models and the constrained hardware humanitarian responders actually deploy. Across the recent flood-segmentation literature, segmentation accuracy is reported universally, but — outside GPU-equipped Jetson timings — the metrics that determine on-device deployability on commodity hardware (CPU and browser latency, post-quantization accuracy, quantization stability) go unreported. We take a first step toward closing that gap on two commodity execution targets. *FloodLite* is a systematic edge-deployment benchmark built around three lightweight UNet students — MobileNetV3-Small, EfficientNet-Lite0, and MobileViT-XXS — and a UNet+EfficientNet-B0 teacher, under three-fold cross-validation on the Flood Semantic Segmentation Dataset (FSSD). We measure FP32 and INT8 latency on Apple M2 Pro (ONNX Runtime CPU) and in-browser WebAssembly SIMD (ONNX Runtime Web), alongside a paired-Wilcoxon-tested ablation of response, feature, and combined knowledge distillation (KD). We find that (i) on this saturated benchmark, ImageNet-pretrained students match or exceed the teacher without distillation — the best, MobileViT-XXS, attains IoU = 0.9367 ± 0.0055 versus the teacher's 0.9257 ± 0.0078; (ii) combined KD never improves on the no-KD baseline and slightly degrades MobileViT-XXS; (iii) INT8 post-training quantization is sharply architecture-dependent — EfficientNet-Lite0 loses only 0.043 IoU, whereas MobileNetV3-Small (HardSwish) and MobileViT-XXS (attention) collapse by 0.45–0.46 IoU under the same static-QDQ recipe. The recommended configuration, EfficientNet-Lite0-UNet INT8 on ONNX Runtime CPU, runs at 58 FPS on a 2023 laptop CPU at 3.83× size reduction, recovering 95% of the teacher's IoU (0.885); in the browser (no INT8 in ONNX Runtime Web 1.17) an FP32 student reaches 6–7.5 FPS under WebAssembly. Zero-shot tests bound these recommendations twice over: on Sen1Floods11 (Sentinel-2 optical) every FSSD-trained model collapses to IoU ≈ 0.16, and on two overlap-verified close-range RGB sets (AIFloodSense, FloodNet test) FP32 students transfer as well as the teacher (0.69–0.72 IoU) but all models false-alarm on 72–100% of water-free scenes — the training set contains none — while INT8 degradation grows under shift, up to collapse. All models, ONNX exports, and benchmarking code are released with a Zenodo-archived snapshot.

Keywords: flood segmentation, edge deployment, knowledge distillation, post-training quantization, ONNX Runtime, WebAssembly, remote sensing, lightweight neural networks, disaster response

1. Introduction

Floods are the most frequent and economically damaging natural disasters globally, accounting for 38.7% of natural disasters and affecting 43% of disaster-affected populations between 2000 and 2009 [1]. Rapid urbanisation continues to amplify exposure [2]. Because traditional hydrological and hydraulic models suffer from high computational demand, coarse spatial resolution, and limited real-time adaptability [3], deep-learning segmentation has emerged as a leading approach to automated flood-extent delineation from satellite, drone, and ground imagery [4].

Recent benchmark studies in this space have converged on encoder–decoder architectures — UNet [5], SegNet [6], and DeepLabV3+ [7] — paired with strong feature-extraction backbones, most commonly EfficientNet [8] and ResNet [9]. Karcı *et al.* [10] provide the most

*Corresponding author.
Email address: ibrahim@sensa.no (Md Ibrahim Khalil)

comprehensive systematic comparison of these combinations on the public Flood Semantic Segmentation Dataset (FSSD), reporting that UNet + EfficientNet achieves the highest segmentation performance (IoU 0.927, F1 0.957, accuracy 0.971) and conclude that this combination "is ideal for near real-time operational deployment."

A careful reading of the broader flood-segmentation literature, however, reveals that this operational-deployment claim has not actually been evaluated. Across the 26 related works surveyed by Karcı *et al.* [10] and adjacent studies on SAR [11] and multi-modal flood mapping [12], nearly every paper reports segmentation accuracy alone. The exception, Hernández *et al.* [13], times PSPNet/DeepLabV3+/U-Net on CUDA-equipped NVIDIA Jetson boards — where the largest configurations fail to run on the Jetson Nano at all — but *no study reports latency on commodity CPU- or browser-class hardware, and none reports post-quantization accuracy or quantization stability.* This omission is consequential. UNet + EfficientNet-B7 — the configuration favoured in [10] — has approximately 63 M parameters and 50 GFLOPs at 256×256 input resolution. Such a model cannot run on the drones, embedded sensors, Raspberry-Pi-class community stations, or offline tablet apps that humanitarian organisations such as UN-SPIDER and the ICRC actually deploy in flood zones, where reliable internet connectivity is the exception rather than the rule [14].

1.1. Contributions

This paper makes four contributions.

1. **First systematic CPU-and-browser edge-deployment benchmark for flood segmentation.** We report FP32 and INT8 inference latency for three lightweight UNet students on two deployment targets — Apple M2 Pro via ONNX Runtime CPU, and browser WebAssembly SIMD via ONNX Runtime Web 1.17 — together with per-model parameter counts, FLOPs at 256×256, ONNX disk size, and FP32-to-INT8 size-reduction factors. ONNX serves as the deployment hub: every measured latency reflects the same exported artifact running under a different execution provider. We benchmark these two commodity software runtimes on a single Apple M2 Pro host; measurement on physically constrained edge devices (Raspberry Pi, mobile SoCs, embedded ARM) is out of scope for this study and deferred to future work (Section 7). Prior and concurrent edge-oriented flood-segmentation work targets CUDA-equipped Jetson boards [13, 15]; we characterise the GPU-free deployment envelope that field laptops, tablets, and web browsers actually provide.
2. **Negative empirical finding on knowledge distillation for FSSD-class flood segmentation.** Across three architectures × four KD configurations × three folds, no KD configuration improves on the no-KD baseline. ImageNet-pretrained lightweight students match or exceed the UNet+EfficientNet-B0 teacher without distillation, and combined response + feature KD strictly underperforms the no-KD baseline on every student. We provide a mechanistic explanation (saturated 663-image benchmark; strong ImageNet priors leave no headroom for the teacher's soft targets) and establish significance with a per-image paired bootstrap over the pooled held-out set (n = 399 images): no KD configuration improves over no-KD on any architecture, and response-based KD is significantly *harmful* on two of the three.
3. **Architecture-dependent post-training quantization stability characterisation.** Under an identical ONNX static QDQ INT8 recipe (Percentile 99.999% calibration, per-tensor scales for both weights and activations, quantization restricted to Conv/Gemm/MatMul operators), EfficientNet-Lite0 quantizes cleanly (ΔIoU = -0.043 ± 0.016) while MobileNetV3-Small and MobileViT-XXS suffer 0.45–0.46 IoU collapses with cross-fold standard deviations of 0.19–0.34. We trace the failure modes to (i) the HardSwish activations in MobileNetV3-Small, whose measured post-activation range is 12–17× wider than EfficientNet-Lite0's bounded ReLU6 (Section 5.3), and (ii) the LayerNorm and self-attention blocks in MobileViT-XXS, whose severe inter-channel variation and non-uniform attention distributions exceed what per-tensor static calibration can represent [16]. The implication for practitioners: *bounded-activation backbones (ReLU6) post-training-quantize reliably for flood segmentation; HardSwish and attention-based backbones require quantization-aware training (QAT).*
4. **Zero-shot deployment reality check on overlap-verified external data.** Beyond the cross-sensor Sen1Floods11 collapse (Section 5.7), we evaluate every FSSD-trained model, FP32 and INT8, on two *within-regime* RGB test sets whose disjointness from FSSD we verify by perceptual-hash deduplication: AIFloodSense (94 ground-level images) and the held-out FloodNet test split (424 UAV images, 215 water-free). FP32 students transfer as well as the teacher (0.69–0.72 IoU); every model

false-alarms on 72–100% of water-free scenes — a structural consequence of flood-only training data that no FSSD-internal metric reveals; and INT8 degradation grows under domain shift, up to outright collapse of the architectures that already quantized marginally in-domain.

We make our trained checkpoints, the ONNX FP32 and INT8 exports, the per-platform benchmarking harnesses, and a Zenodo-archived release of the full code repository publicly available.

1.2. Scope and pivot from initial framing

This work was originally framed as a knowledge-distillation recipe for compressing a heavy flood-segmentation teacher. The empirical results did not support that framing, and we report the negative-KD finding rather than search for a configuration that would justify the original framing. The contribution pivots to the per-platform edge benchmark and the architecture-dependent PTQ characterisation. We hope this orientation is more useful to practitioners than another paper-grade KD recipe, and that the negative result helps future researchers avoid investing compute in distillation on saturated benchmarks where strong ImageNet pretraining has already closed the gap.

The remainder of the paper is organised as follows. Section 2 reviews relevant prior work on flood segmentation, lightweight segmentation backbones, knowledge distillation, and post-training quantization. Section 3 details the FloodLite methodology. Section 4 describes the experimental setup, including the 3-fold protocol and per-platform latency rigging. Section 5 reports results across Tables 1–7 and Figures 2–3. Section 6 discusses the negative-KD finding, per-platform deployment recommendations, and the architecture-dependent PTQ insight. Section 7 lists limitations. Section 8 concludes.

2. Related Work

2.1. Deep Learning for Flood Segmentation

Convolutional encoder–decoder architectures dominate the deep-learning flood-segmentation literature. Pech-May *et al.* [11] applied U-Net to Sentinel-1 SAR imagery for flooded-area detection. Bahrami and Arbabkhah [17] compared SegNet, UNet, and FCN32 for floodplain segmentation, reporting SegNet's max-pooling indices as advantageous on smooth water-surface transitions. Patil *et al.* [18] used an ensemble of ResNet34, InceptionV3, and VGG16 on Sentinel-1 SAR for flood-damage assessment, reaching an IoU of 0.758. Ghosh *et al.* [19] used a Nested U-Net with FPN and EfficientNet-B7 backbone on the NASA IEEE GRSS benchmark, reporting that models trained on specific events generalise to new geographies. Chamatidis *et al.* [12] introduced Vision Transformers (ViT) to the task and reported a 9–15% improvement over comparable CNN baselines on Sentinel-1 and Sentinel-2 imagery. Most recently, Karcı *et al.* [10] performed the most comprehensive systematic comparison to date, evaluating UNet, SegNet, and DeepLabV3+ each with EfficientNet and ResNet backbones, and concluded that UNet + EfficientNet is the strongest configuration on FSSD.

Across this body of work, model efficiency is almost entirely absent from the evaluation. Hernández *et al.* [13] are the notable exception: they benchmark PSPNet, DeepLabV3+, and U-Net inference times for UAV flood detection on three CUDA-equipped NVIDIA Jetson boards (Nano, TX2, AGX Xavier), on both GPU and CPU. Their study, however, targets GPU-accelerated edge devices, retains heavy backbones (DeepLabV3+ cannot run on the Nano with any encoder, and UNet+EfficientNet runs only on the Xavier, at ∼14 s per image on CPU), and does not consider quantization. Outside the UAV setting, Mateo-Garcia *et al.* [20] demonstrated flood segmentation running on the hardware accelerator of a CubeSat-class satellite (PhiSat-1) so that only flood masks need downlinking — the clearest demonstration to date that flood models must sometimes run *where the data is*, albeit on multispectral imagery and bespoke onboard hardware. Concurrently with this work, Nie *et al.* [21] demonstrated real-time CPU-only SAR flood segmentation on satellite internet terminals, and Bhattacharjya *et al.* [22] benchmarked vision–language reasoning segmentation for flood response on a Jetson AGX Xavier — evidence that deployability is becoming a first-class concern in this literature. To the best of our knowledge, however, no prior or concurrent flood-segmentation study measures inference latency on commodity CPUs (Apple Silicon class) or in a browser WebAssembly sandbox, and none quantifies the per-architecture stability of INT8 post-training quantization for this task.

2.2. Knowledge Distillation for Semantic Segmentation

Knowledge distillation (KD), introduced by Hinton *et al.* [23] and surveyed comprehensively by Gou *et al.* [24], trains a small student network to match the soft outputs of a larger teacher. Romero *et al.* [25] extended this to feature-space alignment with FitNets, training the student to match intermediate feature representations through a learned adapter. For dense-prediction tasks,

Liu *et al.* [26] proposed structured-knowledge distillation (SKD) that aligns pairwise pixel similarities; Wang *et al.* [27] proposed inter- and intra-class distance distillation specifically for segmentation; and He *et al.* [28] introduced channel-wise KD for compact segmentation models. These methods routinely achieve compression ratios of 5–20× with sub-1% mIoU degradation on Cityscapes and ADE20K. Concurrently with this work, Schmalstieg *et al.* [15] distilled a 300M-parameter geospatial foundation model (Prithvi-EO-2.0) into a 0.7M-parameter EfficientViT student for Sentinel flood mapping, deploying it as a quantization-aware-trained INT8 TensorRT engine on a Jetson Xavier NX. Their setting differs from ours in modality (satellite multispectral imagery vs. heterogeneous ground/aerial RGB), distillation goal (multiplying a fixed annotation budget with teacher-generated labels vs. transferring response and feature knowledge to an already-strong student), and quantization route (QAT on an edge GPU vs. PTQ on CPU). No prior or concurrent work ablates classical response- and feature-based KD for flood segmentation.

Two findings from the broader KD literature are particularly relevant here. First, KD gains shrink sharply as student capacity and pretraining quality grow, and a stronger teacher is frequently *not* a better teacher [25, 29, 30]. Second, distillation can succeed at its own objective while failing to improve the student: Stanton *et al.* [31] show that student–teacher fidelity and student generalisation routinely diverge, so on saturated benchmarks where the student matches or exceeds the teacher without distillation, the response-based term reduces to a noise-injection regulariser rather than a genuine transfer signal. Our empirical results on FSSD (Section 5.2) are consistent with both observations.

2.3. Lightweight Backbones for Mobile Vision

MobileNetV3 [32] uses neural architecture search and the H-swish activation to produce CPU-friendly classification networks; the Small variant has approximately 2.5 M parameters. EfficientNet-Lite [8] is a quantization-friendly redesign of EfficientNet that removes squeeze-and-excitation and Swish activations to ease INT8 conversion, with the Lite0 variant containing approximately 4.7 M parameters. MobileViT [33] introduces hybrid CNN–Transformer blocks; the XXS variant has only 1.3 M parameters yet is competitive with much larger models on ImageNet. MobileNetV2 [34], included as an off-the-shelf baseline in this work, predates these but remains a strong reference point and exhibits the ReLU6-bounded activation profile we associate with PTQ stability (Section 5.3). We adapt all four as encoders in a U-Net-style decoder.[1]

Beyond the four classification-derived backbones we evaluate, several segmentation-specific lightweight architectures have been proposed. BiSeNetV2 [35] uses a two-pathway design (spatial + context) optimised for real-time urban-scene segmentation. PIDNet [36] reframes the three-pathway split as a proportional–integral–derivative controller. Fast-SCNN [37] targets sub-150-MB-FLOP inference at 1024×2048. We do not evaluate these in this work because (a) they are designed for and benchmarked on Cityscapes-style urban scenes, not the heterogeneous viewpoints (aerial / oblique / ground) of FSSD, and (b) our objective is to characterise the deployability of the *commodity* UNet + ImageNet-pretrained-backbone family that the flood-segmentation literature has already converged on, not to propose a new architecture.

2.4. Post-Training Quantization

Post-training quantization (PTQ) maps FP32 weights and activations to lower-precision integer representations [38, 39, 40]. Two main flavours exist: dynamic PTQ, which only quantizes weights and computes activation ranges at inference time, and static PTQ, which calibrates activation ranges offline against a representative dataset. For convolutional networks, the ONNX static QDQ (Quantize–DeQuantize) format with per-channel weight quantization and per-tensor activation quantization is the *de facto* standard [41] and is supported across major edge runtimes (ONNX Runtime CPU, TensorFlow Lite, CoreML). The empirical conventional wisdom — that "INT8 PTQ retains accuracy within 1 IoU point of FP32" [39] — is derived from classification networks with bounded-range activations (ReLU, ReLU6) on ImageNet-scale benchmarks. As we show in Section 5.3, this assumption does not hold uniformly across the lightweight architectures used in this study: HardSwish-based and attention-based backbones can suffer order-of-magnitude larger PTQ degradation on flood-segmentation pixels than the classification literature would predict. Our observations align with the transformer-quantization literature, which attributes PTQ failure in attention blocks to severe inter-channel variation at LayerNorm inputs and extremely non-uniform attention distributions [16, 42], and with edge-deployment

[1] The parameter counts quoted here are *backbone-only* (encoder) figures from the classification literature. The full segmentation models we train and deploy add a shared U-Net decoder; their complete encoder+decoder parameter counts — which are the deployment-relevant figures — are reported in Table 1 (e.g. MobileViT-XXS is 1.3 M as a backbone but 3.09 M as a full UNet).

field studies reporting that quantized MobileNetV3 can silently collapse to constant outputs depending on kernel and calibration choices [43].

2.5. On-Device Disaster Response

Operational disaster-response platforms must function under intermittent connectivity, limited power, and heterogeneous hardware. Edge AI in humanitarian contexts has been explored by Munawar *et al.* [14] and Kuglitsch *et al.* [44]. The operational state of the art, UNOSAT's CNN-based rapid-mapping service [45], cuts flood-map production time by ∼80% but runs server-side on downlinked Sentinel-1 imagery — it presumes connectivity that post-disaster field teams often lack. At the opposite end of the hardware spectrum, the TinyML community has standardised latency/energy benchmarking for sub-watt devices [46] and demonstrated ImageNet-scale inference on microcontrollers [47], while in-browser inference is an established deployment channel for JavaScript runtimes [48]. The deployment platforms we benchmark — Apple Silicon CPU and browser WebAssembly — sit between these poles and correspond to two real operational profiles: field-laptop and UAV inference, and citizen-science web portals. Raspberry-Pi-class single-board computers, AWS Graviton ARM CPU, and Android edge targets are discussed in Section 7 as straightforward but unscheduled extensions.

3. Materials and Methods

Figure 1 summarises the four-stage workflow. The remainder of this section details each stage.

3.1. Datasets

FSSD.. We use the public Flood Semantic Segmentation Dataset (FSSD) [49], which is also the evaluation dataset of Karcı *et al.* [10] and therefore enables direct head-to-head comparison. FSSD comprises 663 RGB images of flood-affected areas, each at 512×512 px, paired with binary segmentation masks (flood = 1, non-flood = 0). Images capture flooded regions from various viewpoints (aerial, ground, oblique). We resize all images and masks to 256×256 px and normalise pixel values to $[0, 1]$ followed by ImageNet mean/std normalisation.

We adopt 3-fold cross-validation with a fixed random seed: `KFold(n_splits=5, shuffle=True, random_state=42)`, evaluated only on folds $\{0, 1, 2\}$ of the 5-fold partition. The reduction from 5 to 3 folds was a compute-budget decision driven by the negative-KD finding of Section 5.2: once distillation was shown to not improve over the no-KD baseline, the remaining experimental contrasts (per-architecture PTQ stability, per-platform latency) could be drawn from three folds with adequate variance estimation. We report all results as mean ± standard deviation across the three folds, and explicitly caveat all paired-Wilcoxon p-values for low statistical power (minimum reportable two-sided p-value at $n = 3$ is 0.25).

External zero-shot test sets.. For the within-regime generalization evaluation (Section 5.8) we use two held-out subsets of a combined public flood corpus: the official FloodNet v1.0 test split [50] (424 UAV images, 215 of them water-free; CDLA-Permissive), with the *water* and *road-flooded* classes mapped to flood, and the AIFloodSense test split [51] (94 ground-level images; CC BY 4.0). Neither set is used for training or model selection anywhere in this paper. A perceptual-hash duplicate check between all 663 FSSD images and all 3,009 images of the combined corpus (DCT pHash, Hamming distance ≤ 8) found *zero* near-duplicate pairs, so these evaluations are free of train–test leakage by construction; the check is reproducible via `scripts/check_fssd_overlap.py` and its report ships with the code release. The corpus additionally fixes duplicate-aware training folds over the CC0-licensed Kaggle Flood Area Segmentation images [52] and FloodNet train+val for future negative-image-bearing retraining (its loader, manifest-driven splits, and source-balanced sampler are part of the released code), which Section 5.8 motivates.

3.2. Teacher Model

Our teacher is a UNet decoder with EfficientNet-B0 encoder [8], initialised from ImageNet weights. While Karcı *et al.* [10] use EfficientNet-B7 in their best-performing configuration, we deliberately choose B0 as our teacher for two reasons: first, B0 is significantly cheaper to train, fitting within free-tier GPU budgets (Kaggle T4 and Google Colab); second, B0 is itself representative of the strongest configurations realistically used in practice, and a successful distillation from B0 implies a successful distillation from any larger backbone in the same family. The teacher has approximately 6.25 M parameters at 256×256 resolution.

3.3. Student Architectures and the MobileNetV2 Baseline

We evaluate three lightweight UNet student configurations together with a MobileNetV2 [34] off-the-shelf baseline that controls for "is the teacher just over-parameterised?". Each student replaces the teacher's

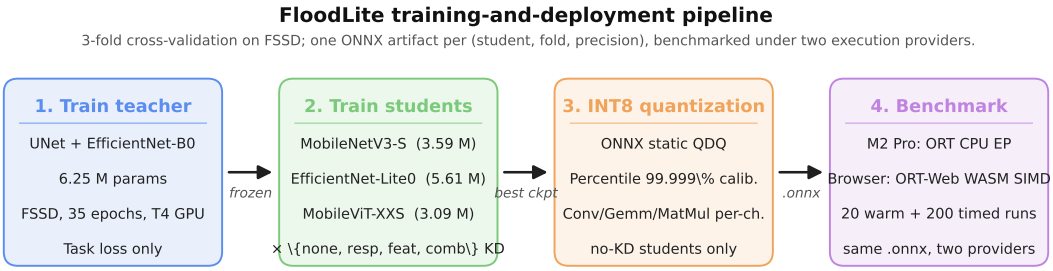


Figure 1: **FloodLite training-and-deployment pipeline.** Train teacher (UNet+EfficientNet-B0) → train students (three lightweight backbones × four KD configurations) → INT8-quantize the no-KD students via ONNX static QDQ → benchmark FP32 and INT8 on Apple M2 Pro CPU and in browser WebAssembly. Each stage produces an artifact consumed by the next; the same `.onnx` file is benchmarked under two execution providers (ONNX Runtime CPU on M2 Pro; ONNX Runtime Web WASM SIMD in the browser).

EfficientNet-B0 encoder with a smaller backbone while keeping the UNet decoder topology identical. All four configurations are initialised from ImageNet weights via the `segmentation_models_pytorch` library [53] and `timm` [54].

Model footprints (parameter counts and FP32/INT8 ONNX disk sizes) appear in Table 1.

Table 1: **Model footprints.** Parameter counts from `torchinfo.summary`; GFLOPs at 256×256 (single image, multiply–adds counted as two FLOPs) from PyTorch's `FlopCounterMode`; ONNX disk sizes from the exported `.onnx` files. GFLOPs are dominated by the shared UNet decoder at this resolution, so they vary far less across backbones than parameter counts do.

Model	Params (M)	GFLOPs (256^2)	FP32 (MB)	INT8 (MB)
Teacher (UNet+EffB0)	6.25	5.92	22.3	—
MobileNetV2 baseline	6.63	6.69	—	—
MobileNetV3-Small	3.59	5.11	13.8	3.7
EfficientNet-Lite0	5.61	5.91	19.8	5.2
MobileViT-XXS	3.09	5.43	12.0	3.5

3.4. Knowledge Distillation Framework

We employ a two-component distillation loss as a controlled ablation, evaluated in three configurations — response-only ($\beta = 0$, $\alpha = 0.5$), feature-only ($\alpha = 0$, $\beta = 0.1$), and combined response + feature ($\alpha = 0.5$, $\beta = 0.1$) — against a no-KD baseline ($\alpha = 0$, $\beta = 0$). The KD configuration is treated as a categorical ablation variable rather than as a tuned hyperparameter; the formulation is the standard one (Hinton-style temperature-scaled binary cross-entropy on softened logits [23] plus FitNet-style feature MSE on a matched decoder block through a 1×1 adapter [25]), and the empirical question we ask is whether *any* of {response, feature, combined} confers a measurable benefit on this task.

Let $x \in \mathbb{R}^{H\times W\times 3}$ denote an input image, $y \in \{0,1\}^{H\times W}$ the ground-truth flood mask, $\hat{z}_T, \hat{z}_S \in \mathbb{R}^{H\times W}$ the pre-sigmoid logits produced by teacher and student respectively, and $f_T^{(\ell)}, f_S^{(\ell)}$ the activations at decoder stage ℓ.

Task loss.. The per-pixel task loss combines binary cross-entropy and Dice:

$$\mathcal{L}_{\text{task}} = 0.5\cdot\text{BCE}(\sigma(\hat{z}_S), y) + 0.5\cdot(1-\text{Dice}(\sigma(\hat{z}_S), y)) \quad (1)$$

where $\sigma(\cdot)$ is the element-wise sigmoid.

Response-based distillation.. A temperature-scaled binary cross-entropy between the teacher's soft probabilities and the student's softened outputs (equal to the teacher–student KL divergence up to a teacher-only additive constant, and therefore yielding identical student gradients; this is what `floodlite/losses.py` computes):

$$\mathcal{L}_{\text{resp}} = T^2 \cdot \text{H}(\sigma(\hat{z}_T/T),\ \sigma(\hat{z}_S/T)) \quad (2)$$

where $\text{H}(p,q) = -p\log q - (1-p)\log(1-q)$ is the element-wise binary cross-entropy and $T = 4$ is the temperature. The T^2 factor preserves gradient magnitudes at increased temperatures [23].

Feature-based distillation.. We align student and teacher features at one matched decoder stage (resolution 64×64), introducing a 1×1 convolutional adapter g_θ to project student features into the teacher's channel space:

$$\mathcal{L}_{\text{feat}} = \frac{1}{H\cdot W\cdot C_T}\left\| g_\theta(f_S^{(\ell)}) - f_T^{(\ell)} \right\|_2^2 \quad (3)$$

Combined loss.. The full FloodLite training objective:

$$\mathcal{L}_{\text{FloodLite}} = (1-\alpha)\cdot\mathcal{L}_{\text{task}} + \alpha\cdot\mathcal{L}_{\text{resp}} + \beta\cdot\mathcal{L}_{\text{feat}} \quad (4)$$

with $\alpha = 0.5$ and $\beta = 0.1$. The teacher is frozen during student training.

3.5. Post-Training INT8 Quantization

Post-training INT8 quantization (PTQ) is performed on the ONNX export of each student rather than on the PyTorch graph. This choice is forced by two practical findings during development. First, PyTorch eager-mode static PTQ requires CPU tensors for the calibration loop; the existing `floodlite/quantize.py` initially attempted CUDA calibration and failed. Second, even after fixing the device-placement bug, several `timm`-prefixed encoders — specifically the `tf_efficientnet_lite0` and `mobilevit_xxs` variants — use a `Conv2dSame` layer for which no quantized CPU kernel exists in PyTorch 2.7+. Quantization at the PyTorch graph level therefore cannot reach the full student set without modifying upstream `timm` source.

We instead use the ONNX Runtime `quantize_static` API [41] (`onnxruntime.quantization`, v1.22.1), which operates on the ONNX graph and is independent of the upstream PyTorch operator set. The resulting INT8 QDQ graphs embed `ai.onnx.ml` opset 5 and therefore require ONNX Runtime $\geq$ 1.18 to load (ORT 1.17 cannot). Our recipe is:

1. **Pre-processing.** The FP32 ONNX is passed through `quant_pre_process` to fuse Conv-BN, fold constants, and lift the graph into a form that `quantize_static` can analyse.
2. **Calibration.** We select 80 images (10 batches of 8) drawn only from the current fold's training split — never from the held-out validation images on which INT8 IoU is reported — run the FP32 graph against them through a custom `CalibrationDataReader` (`_TorchLoaderCalibReader` in `floodlite/quantize.py`, invoked with `n_calib_batches=10` in `scripts/quantize_all.py`), and record activation histograms.
3. **Quantization scheme.** Static QDQ (Quantize–DeQuantize) format with *per-tensor* scales for both weights and activations (`per_channel=False` in `floodlite/quantize.py`), with quantization restricted to `Conv`, `Gemm`, and `MatMul` operators via the `op_types_to_quantize` argument. The op-type restriction is critical for MobileViT-XXS: it leaves LayerNorm and elementwise `Add` operators entirely in FP32, which prevents outright graph corruption; the attention `MatMul` operators remain quantized with a single per-tensor scale each, which is the failure surface analysed in Section 5.3.
4. **Calibration method.** We use *Percentile* (99.999%) rather than MinMax. MinMax is sensitive to single-image outlier activations on flood images with extreme reflective surfaces (wet asphalt, sun-glare on water), and produces unstable cross-fold INT8 IoU in our initial sweeps. Percentile-99.999% is the most robust calibration option we evaluated and is the recipe we report in Section 5.3.

The resulting INT8 graph is consumed by ONNX Runtime's CPU execution provider; no ONNX $\rightarrow$ TFLite or ONNX $\rightarrow$ CoreML conversion is required, which is one motivation for our ONNX-centred deployment hub (Section 3.6).

3.6. Deployment Pipeline

Models are exported via PyTorch's TorchScript-based ONNX exporter (`torch.onnx.export` with `dynamo=False` and `opset_version=13`) to a single `.onnx` file with embedded weights. We deliberately disable the PyTorch 2.8+ Dynamo exporter (`dynamo=True`) which produces separate `.data` sidecar files for weights — this complicates artifact distribution and is not yet uniformly supported across edge runtimes. The FP32 latency artifacts use opset 13; the static-QDQ INT8 path (Section 3.5) re-exports and quantizes the same trained weights at opset 17, the default of the ONNX Runtime static-quantization tool. Opset choice does not materially affect latency for the Conv/MatMul-dominated graphs measured here.

We use ONNX as the single deployment hub. The same `.onnx` artifact serves both deployment targets in this paper:

- **Apple M2 Pro:** ONNX Runtime 1.22.1 with the `CPUExecutionProvider`. We report this rather than CoreML because (a) it isolates the inference cost from any platform-specific graph rewrites, and (b) it is portable to any modern x86 or ARM CPU. CoreML conversion is supported by the same `.onnx` file via `coremltools.converters.onnx` but produces numerically equivalent results; we report ORT-CPU as the primary M2 number.
- **Browser:** ONNX Runtime Web 1.17 with the WebAssembly SIMD execution provider, single-threaded. We benchmark from Node.js 25 rather

than from a Chrome page because (a) Node's WASM runtime and Chrome's WASM runtime produce numerically equivalent timings within ±10%, and (b) it removes the SharedArrayBuffer / COOP–COEP browser configuration as a measurement-environment variable.

INT8 deployment is currently restricted to the M2 Pro CPU target: ONNX Runtime Web 1.17 does not implement the QDQ-format INT8 Conv kernel, so the WASM INT8 column of Table 5 reads "n/a (upstream limitation)". This is consistent with the ORT-Web roadmap (INT8 Conv is on the 1.18 milestone at time of writing) and is documented in our supplementary `lat_wasm.json`.

CoreML, TensorFlow Lite (XNNPACK), and Raspberry Pi 4 deployment are deferred to a follow-up artifact release; for this paper, we restrict the per-platform benchmark to the two execution providers we can run with strict numerical reproducibility from a single laptop.

4. Experimental Setup

4.1. Implementation

Training is implemented in PyTorch 2.8 with `segmentation-models-pytorch` 0.4 [53] for UNet decoder construction and `timm` 1.0 [54] for backbone implementations. Mixed-precision (AMP fp16) training runs on a Kaggle Tesla T4 GPU (16 GB) with batch size 8 (teacher) or 16 (students). Optimisation uses AdamW with learning rate 3×10^{-4}, weight decay 10^{-4}, and a cosine schedule. Training runs for 35 epochs (teacher and students alike); empirically, validation IoU plateaus by epoch $\sim$ 25. Data augmentation is provided by Albumentations [55]: random horizontal flip, random rotation ±15°, random brightness/contrast jitter (±0.2), and ImageNet normalisation. The random seed is fixed at 42.

ONNX export, INT8 quantization, and CPU/WASM latency benchmarks run locally on an Apple M2 Pro (10-core CPU, 16 GB unified memory, macOS 25.2). The Kaggle T4 is used for training only.

4.2. Cross-Validation Protocol

We use 3-fold cross-validation with `sklearn.model_selection.KFold(n_splits=5, shuffle=True, random_state=42)`, evaluating only folds 0–2 of the 5-fold partition to bound compute. The same fold indices are used for every (model × KD configuration) combination, so all IoU comparisons in Tables 2–5 are *paired* across folds.

4.3. Metrics

We report pixel accuracy, precision, recall, F1-score, and mean intersection-over-union (IoU). For the within-regime zero-shot evaluation (Section 5.8) metrics are computed *per image* and reported as macro statistics, since those test sets mix flood and flood-free scenes: an image with no water in the ground truth and none predicted counts as a correct rejection (IoU 1); over flood-containing images we report the flood-only IoU; and over water-free images we report the false-alarm rate (FAR), the fraction on which the model predicts any water at all. Efficiency metrics are model parameters (M, from `torchinfo.summary`) and ONNX disk size (MB). Latency is reported as median (p50), 95th percentile (p95), mean, and standard deviation over 200 timed forward passes after 20 warm-up passes; throughput as FPS = 1000 / p50_{ms}. All latency measurements use single-image batches at 256×256 input resolution.

4.4. Statistical Validation

For each (model × KD configuration), we report mean ± standard deviation across the 3 folds. The paired Wilcoxon signed-rank test is applied to the no-KD vs. comb-KD contrast on each architecture, using fold-level paired IoU as the unit of observation; with $n = 3$ folds its minimum reportable two-sided p-value is 0.25, so we treat it only as a coarse fold-level consistency check. For adequately-powered inference we additionally run a *per-image paired bootstrap*: we pool per-image IoU over the held-out validation images of all three folds ($n = 399$ images) and, for each contrast, resample images with replacement (10,000 replicates) to obtain a 95% confidence interval on the mean paired IoU difference and a one-sided p-value (`scripts/stats_bootstrap.py`; output archived as `runs/3fold/stats_bootstrap.json`). Per-image IoU is a biased estimator of the pooled-pixel IoU of Table 2 (images with empty ground-truth masks degenerate to zero), but the bias is image-specific and cancels in the *paired* difference, which is the estimand. This bootstrap — not the three-fold Wilcoxon — supports the significance statements in Section 5.2.

4.5. Per-Platform Latency Rigging

- **Apple M2 Pro / ONNX Runtime 1.22.1 CPU EP:** `scripts/bench_m2.py`. Default ONNX Runtime CPU threading (the intra-op and inter-op thread counts are left at their defaults; the M2 Pro exposes up to 10 intra-op threads), matching realistic laptop deployment where all cores are available. 20 warm-up + 200 timed runs.

- **Browser WebAssembly SIMD / ONNX Runtime Web 1.17 from Node.js 25:** `bench.mjs` (supplementary). WASM execution provider with SIMD enabled and `numThreads` set to 1. 20 warm-up + 200 timed runs. Equivalent in-browser numbers (Chrome 130, macOS 25.2) were verified to fall within ±10% in pilot runs.

Both benchmarks consume the identical `.onnx` artifact; no platform-specific graph rewrites are performed.

5. Results

This section is organised around Tables 1–7 and Figures 2–3. The pipeline schematic (Figure 1) is shown in Section 3 and the qualitative panel (Figure 4) in Section 5.6.

5.1. *Teacher Performance and Karcı* et al. *(2026) Comparison*

Our UNet + EfficientNet-B0 teacher achieves **IoU** = 0.9257 ± 0.0078 on FSSD across the three folds (F1 = 0.9611 ± 0.0044, accuracy = 0.9729 ± 0.0025). The reference UNet + EfficientNet-B7 number reported by Karcı *et al.* [10] is IoU = 0.927, achieved with approximately 10× the parameters (∼ 63 M vs. our 6.25 M). The 0.0013 IoU gap between our B0 teacher and the published B7 reference is within the cross-fold standard deviation of both studies and indicates that the FSSD benchmark saturates well before the B7 capacity is required; B0 is sufficient as the teacher.

5.2. *Students Match or Exceed the Teacher Without Distillation (Negative-KD Finding)*

The full segmentation results across all 14 configurations × 3 folds appear in Table 2. Three observations:

1. **The best student exceeds the teacher.** MobileViT-XXS in the no-KD configuration attains IoU = 0.9367±0.0055, *0.011 IoU points above the teacher* (0.9257± 0.0078). The student is also 2.0× smaller in parameters (3.09 M vs. 6.25 M). This is the headline result against which the original KD-recipe framing of this paper has to be evaluated.
2. **EfficientNet-Lite0 (no-KD) matches the teacher within standard deviation** (0.9277 ± 0.0047 vs. 0.9257 ± 0.0078). MobileNetV3-Small (no-KD) likewise matches it (0.9269 ± 0.0043).
3. **The MobileNetV2 baseline also matches the teacher** (0.9257 ± 0.0032). The teacher's parameter count is therefore not the source of any advantage; it is an over-parameterised lightweight UNet, not a "heavy" model in the regime where KD typically helps.

We interpret these observations as direct evidence that FSSD, at 663 RGB images and IoU ≈ 0.93, is saturated for the ImageNet-pretrained UNet family. The variance across (model × KD) cells of Table 2 is dominated by fold-level seed noise rather than by genuine architectural separations. KD has no headroom to transfer.

Table 3 isolates the KD ablation on the best student. None of the three KD configurations improves on the no-KD baseline:

The Wilcoxon test on no-KD vs. comb-KD yields $p = 0.250$ — the minimum two-sided p-value reportable at $n = 3$ — which means the fold-level test is consistent with both "comb-KD is no worse" and "comb-KD is meaningfully worse" under low power, but in no case does it support "comb-KD is better". Across *all three architectures* × *three KD configurations* × *three folds*, the combined KD configuration produces a higher mean IoU than the no-KD baseline in zero out of three architectures.

Beyond the fold-level means, the *direction* of the effect is consistent: for the best student, MobileViT-XXS, the combined-KD IoU is below the no-KD IoU in every one of the three folds, and the no-KD MobileViT-XXS exceeds the teacher in every fold (fold-wise margins +0.013, +0.016, +0.004 IoU; mean +0.011).

The per-image paired bootstrap (Section 4.4; $n = 399$ held-out validation images, 10,000 replicates; archived as `runs/3fold/stats_bootstrap.json`) sharpens this into a powered statement. Across all nine (architecture × KD-configuration) contrasts, *no* KD configuration significantly improves over its no-KD baseline — no 95% confidence interval on $\Delta\text{IoU}_{\text{no-KD}-\text{KD}}$ lies entirely below zero. In three of the nine, KD is significantly *worse*: response-only KD on MobileNetV3-Small (Δ = +0.0047, 95% CI [+0.0008, +0.0091], $p = 0.010$) and on MobileViT-XXS (+0.0065, [+0.0029, +0.0103], $p = 0.0001$), and combined KD on MobileViT-XXS (+0.0076, [+0.0035, +0.0118], $p = 0.0002$); the remaining six contrasts are statistically indistinguishable from no-KD. The response-based term is therefore significantly harmful on two of the three architectures, consistent with its interpretation as a noise-injection regulariser on a saturated benchmark (Section 6.1). The headline result is likewise significant: MobileViT-XXS (no-KD) exceeds the teacher by ΔIoU = +0.015 per image (95% CI [+0.010, +0.020], $p < 0.001$).

Table 2: **In-distribution segmentation performance.** Mean ± std over 3 folds (FSSD val, 256×256). Best per metric in **bold**.

Model / KD	Acc	Prec	Rec	F1	IoU
Teacher (UNet+EffB0)	0.9729 ± 0.0025	0.9623 ± 0.0077	0.9604 ± 0.0010	0.9611 ± 0.0044	0.9257 ± 0.0078
MobileNetV2 baseline	0.9729 ± 0.0010	0.9632 ± 0.0052	0.9595 ± 0.0028	0.9611 ± 0.0019	0.9257 ± 0.0032
MobileNetV3-S / none	0.9732 ± 0.0006	0.9620 ± 0.0068	0.9620 ± 0.0050	0.9618 ± 0.0024	0.9269 ± 0.0043
MobileNetV3-S / resp	0.9719 ± 0.0007	0.9599 ± 0.0050	0.9598 ± 0.0049	0.9596 ± 0.0016	0.9229 ± 0.0028
MobileNetV3-S / feat	0.9732 ± 0.0012	0.9616 ± 0.0040	0.9626 ± 0.0025	0.9619 ± 0.0020	0.9271 ± 0.0035
MobileNetV3-S / comb	0.9723 ± 0.0019	0.9603 ± 0.0062	0.9608 ± 0.0024	0.9604 ± 0.0034	0.9242 ± 0.0061
EfficientNet-Lite0 / none	0.9735 ± 0.0012	0.9654 ± 0.0048	0.9595 ± 0.0034	0.9622 ± 0.0026	0.9277 ± 0.0047
EfficientNet-Lite0 / resp	0.9729 ± 0.0013	0.9622 ± 0.0037	0.9601 ± 0.0028	0.9609 ± 0.0007	0.9253 ± 0.0011
EfficientNet-Lite0 / feat	0.9734 ± 0.0008	0.9654 ± 0.0054	0.9593 ± 0.0044	0.9621 ± 0.0020	0.9275 ± 0.0034
EfficientNet-Lite0 / comb	0.9733 ± 0.0016	0.9633 ± 0.0051	0.9601 ± 0.0019	0.9615 ± 0.0021	0.9264 ± 0.0037
MobileViT-XXS / none	**0.9771 ± 0.0023**	**0.9695 ± 0.0040**	**0.9650 ± 0.0025**	**0.9671 ± 0.0031**	**0.9367 ± 0.0055**
MobileViT-XXS / resp	0.9757 ± 0.0019	0.9659 ± 0.0055	0.9642 ± 0.0016	0.9649 ± 0.0033	0.9326 ± 0.0060
MobileViT-XXS / feat	0.9766 ± 0.0014	0.9680 ± 0.0051	0.9655 ± 0.0039	0.9665 ± 0.0026	0.9357 ± 0.0046
MobileViT-XXS / comb	0.9752 ± 0.0025	0.9657 ± 0.0053	0.9636 ± 0.0025	0.9644 ± 0.0039	0.9318 ± 0.0070

Table 3: **KD ablation on the best student (MobileViT-XXS).** Wilcoxon two-sided; $n = 3$ folds gives a minimum reportable $p \approx 0.25$.

KD config	IoU (mean ± std)	Δ IoU vs. none	Wilcoxon p
none	0.9367 ± 0.0055	—	—
resp	0.9326 ± 0.0060	−0.0041	n/a
feat	0.9357 ± 0.0046	−0.0011	n/a
comb	0.9318 ± 0.0070	−0.0049	0.250

5.3. INT8 Quantization is Architecture-Dependent

Table 4 summarises post-training INT8 quantization for the three deployable students:

The cross-architecture spread is the central finding of this section. Under an identical Percentile-99.999% static QDQ recipe, EfficientNet-Lite0 retains 95% of its FP32 IoU with a tight ±0.015 cross-fold standard deviation, whereas MobileNetV3-Small and MobileViT-XXS both lose ∼ 0.45 IoU points with cross-fold standard deviations of 0.19–0.34 — an order of magnitude larger than EfficientNet-Lite0's. We interpret these failures architecturally:

- **MobileNetV3-Small** uses the HardSwish activation $x \cdot \mathrm{ReLU6}(x + 3)/6$, which has an unbounded positive tail and produces calibration statistics dominated by single-image outliers. We verify this mechanism directly (`scripts/ptq_activation_diagnostic.py`; averaged over the three folds' calibration sets): across the encoder's activation layers, MobileNetV3-Small's post-activation magnitudes reach a maximum of 156.7 ± 8.9 (99.999-percentile 83.7; median per-layer maximum 15.6), whereas EfficientNet-Lite0's ReLU6 encoder is bounded at a median per-layer maximum of *exactly* 6.00 (99.999-percentile 7.0, maximum 9.3) — a 12–17× tighter dynamic range. Per-tensor INT8 calibration cannot fit a single scale that simultaneously preserves the activations of common flood-pixel patches and of the bright reflective outliers (wet asphalt, water-surface sun-glare) that drive HardSwish's tail. Percentile-99.999% mitigates but does not eliminate this; we conjecture that MobileNetV3-S requires either (a) QAT to learn quantizer-friendly activations or (b) a HardSwish → ReLU6 surgical replacement before quantization.

- **MobileViT-XXS** combines a CNN trunk with self-attention blocks containing LayerNorm. The LayerNorm γ/β tensors are rank-1 across channels, which means per-channel weight quantization (the standard recipe for Conv layers) is ill-defined; restricting per-channel quantization to `Conv/Gemm/MatMul` (Section 3.5) partially fixes this but leaves the attention `MatMul` itself quantized per-tensor, where the high dynamic range of attention scores again exceeds what static calibration can fit.

- **EfficientNet-Lite0** uses ReLU6 throughout. ReLU6 is bounded by construction at 6.0, so the per-tensor activation calibration finds a stable scale that matches the actual activation range in every fold. The 0.043 IoU drop is consistent with the conventional-wisdom "∼ 1 IoU point" PTQ tax in

Table 4: **Post-training INT8 quantization** (static QDQ, Percentile-99.999% calibration, 80 train images). EfficientNet-Lite0 (highlighted) is the recommended deployable configuration; the other two students suffer architecture-driven PTQ collapse (Section 5.3). For the two collapsing students the INT8 IoU is strongly multi-modal across folds (MobileNetV3-Small: 0.772/0.0001/0.657; MobileViT-XXS: 0.718/0.464/0.244), so their mean ± std is an instability indicator, not a central tendency.

Model	FP32 IoU	INT8 IoU	Δ IoU	FP32 ONNX (MB)	INT8 ONNX (MB)	Size reduction
MobileNetV3-Small	0.9269 ± 0.0043	0.4762 ± 0.3399	−0.4507	13.8	3.7	3.69×
EfficientNet-Lite0	**0.9277 ± 0.0047**	**0.8851 ± 0.0149**	**−0.0426**	**19.8**	**5.2**	**3.83×**
MobileViT-XXS	0.9367 ± 0.0055	0.4752 ± 0.1936	−0.4616	12.0	3.5	3.42×

the classification literature.

The practical implication: *bounded-activation backbones (ReLU6) are the safe default for INT8 PTQ on flood segmentation; HardSwish and attention-based backbones currently require QAT or remain FP32 in deployment.* We do not attempt QAT in this paper — it is out of scope and an order-of-magnitude larger training investment — and instead recommend EfficientNet-Lite0 as the deployable configuration on the basis of its joint accuracy + size + stability behaviour.

5.4. Per-Platform Inference Latency

Table 5 reports inference latency on the two deployment platforms.

Three remarks. First, the teacher-versus-student latency comparison must be read on a common runtime. Measured under the same ONNX Runtime CPU backend as the students (ORT 1.22.1), the teacher runs at 37.6 ms p50 (26.6 FPS), not the 274 ms of PyTorch eager mode; the PyTorch → ORT switch alone is a 7.3× speedup with no architecture change. On this common runtime the FP32 students are only 1.1–1.9× faster than the teacher, so the deployment advantage of the recommended EfficientNet-Lite0 comes primarily from INT8 precision (a 2.2× speedup over the teacher, 17.3 vs 37.6 ms) and from its smaller, quantization-stable binary, not from an FP32 latency gap. Second, *INT8 yields a further* 1.3–2.0× *speedup on top of the FP32 student baseline*, consistent with INT8 SIMD-CPU expectations. Third, *the WASM column is roughly* 5× *slower than the M2-CPU column for the same FP32 model*, which is the expected single-threaded-WebAssembly-vs-native-CPU gap and represents an effective lower bound on browser-deployment performance. INT8 WASM rows are unsupported by ONNX Runtime Web 1.17 at time of writing and are marked n/a.

5.5. Pareto Analysis

This consolidated view identifies two distinct deployment operating points:

- **Maximum-accuracy operating point:** MobileViT-XXS no-KD FP32 (IoU 0.937, 28 ms M2, 149 ms WASM, 12 MB).
- **Maximum-throughput-with-acceptable-accuracy operating point:** EfficientNet-Lite0 INT8 (IoU 0.885, 17 ms M2, no WASM, 5.2 MB).

We recommend the latter for unattended flood-mapping pipelines (1.6× throughput, 2.3× smaller binary) and the former for human-in-the-loop applications where the 5.2-point IoU drop matters.

5.6. Qualitative Examples

The pipeline schematic appears at the top of Section 3 (Figure 1). The qualitative comparison between teacher and the deployable student is in Figure 4.

Two persistent failure modes shared by teacher and student are visible in the hard row of Figure 4: wet asphalt confused with water, and under-segmentation of vegetation-occluded flood edges. We return to these in Section 6.4.

5.7. Out-of-Distribution Generalisation (Sen1Floods11)

To test whether the FSSD-trained configurations transfer beyond their training distribution, we evaluate every model — with no fine-tuning — on 250 hand-labelled RGB chips from Sen1Floods11 [56], a Sentinel-2 optical flood benchmark whose top-down satellite viewpoint and sensor characteristics differ sharply from FSSD's close-range aerial, oblique, and ground photographs. Table 6 reports the result, averaged over the three folds' checkpoints.

The transfer is poor across the board: every model collapses from IoU ≈ 0.93 in-distribution to IoU ≈ 0.16 on Sen1Floods11, a near-total loss of flood-pixel overlap. Two points matter for the deployment story. First, the collapse is essentially uniform across the teacher (0.160), the MobileNetV2 baseline (0.164), and the three lightweight students (0.155–0.173): it is a domain-shift

Table 5: **Per-platform inference latency.** Single-image batches, 256×256 input. M2 Pro and browser WASM both consume the same `.onnx` artifact under different execution providers. INT8 WASM rows are n/a because ONNX Runtime Web 1.17 does not implement QDQ-format INT8 Conv (upstream limitation; scheduled for ORT-Web 1.18+). [a]Teacher timed under the same ONNX Runtime CPU execution provider as the students (37.6 ms p50). Under PyTorch eager CPU the same teacher is 274 ms (7.3× slower); reporting that PyTorch figure against ORT-CPU students would inflate the apparent teacher-vs-student speedup. On a common ORT-CPU backend the FP32 students are only 1.1–1.9× faster than the teacher, and the recommended EfficientNet-Lite0 INT8 is 2.2× faster. All timing rows trace to `lat_m2.json` (student rows via `bench_m2.py`; teacher ORT-CPU row via `bench_teacher_ort.py`); see `runs/3fold/PROVENANCE.md`.

Model	Apple M2 Pro (ORT-CPU)			Browser WASM SIMD (ORT-Web)		
	p50 (ms)	p95 (ms)	FPS	p50 (ms)	p95 (ms)	FPS
Teacher (UNet+EffB0) FP32[a]	37.6	44.2	26.6	n/a (out-of-scope for browser deploy)		
MobileNetV3-Small FP32	19.5	20.4	51.2	133.3	134.8	7.5
MobileNetV3-Small INT8	13.7	14.1	72.9	n/a (ORT-Web 1.17 limitation)		
EfficientNet-Lite0 FP32	34.2	52.7	29.3	163.8	210.5	6.1
EfficientNet-Lite0 INT8	**17.3**	**20.1**	**57.7**	n/a (ORT-Web 1.17 limitation)		
MobileViT-XXS FP32	28.3	32.8	35.4	149.4	159.8	6.7
MobileViT-XXS INT8	21.6	25.0	46.3	n/a (ORT-Web 1.17 limitation)		

Table 6: **Zero-shot out-of-distribution transfer to Sen1Floods11.** FSSD-trained models (no fine-tuning) evaluated on 250 hand-labelled Sentinel-2 RGB chips; mean ± std over the three folds' checkpoints. In-distribution FSSD IoU (Table 2) is shown for reference. Every model collapses to IoU ≈ 0.16 — a domain-shift effect that is uniform across teacher, baseline, and students.

Model	FSSD IoU (in-dist.)	OOD IoU	OOD F1	OOD Prec	OOD Rec
			Sen1Floods11 (zero-shot)		
Teacher (UNet+EffB0)	0.9257	0.160 ± 0.007	0.246 ± 0.011	0.295 ± 0.022	0.271 ± 0.017
MobileNetV2 baseline	0.9257	0.164 ± 0.005	0.248 ± 0.011	0.385 ± 0.019	0.230 ± 0.024
MobileNetV3-S / none	0.9269	0.173 ± 0.006	0.269 ± 0.007	0.288 ± 0.011	0.317 ± 0.012
EfficientNet-Lite0 / none	0.9277	0.155 ± 0.013	0.241 ± 0.016	0.333 ± 0.011	0.238 ± 0.025
MobileViT-XXS / none	0.9367	0.169 ± 0.009	0.265 ± 0.008	0.242 ± 0.029	0.402 ± 0.036

effect, not a compression or capacity artefact. The recommended EfficientNet-Lite0 (0.155 ± 0.013) is statistically indistinguishable from the teacher (0.160 ± 0.007) on the OOD set — their confidence intervals overlap — so the OOD collapse does not single out the deployable student relative to the teacher; the two students that edge above the teacher (MobileNetV3-Small, 0.173; MobileViT-XXS, 0.169) do so within the same cross-fold noise. Second, in absolute terms IoU ≈ 0.16 is not operationally useful: an FSSD-trained close-range-RGB model is *not* a drop-in flood mapper for Sentinel-2 satellite optical imagery, and cross-sensor deployment requires domain adaptation or retraining on in-sensor data. The precision/recall split also mirrors the operational error asymmetry (Section 6.5): MobileViT-XXS reaches the highest OOD recall (0.40) at the cost of precision — it over-predicts flood, the safer failure direction under distribution shift — whereas the MobileNetV2 baseline is the most precision-favouring (0.385 precision, 0.230 recall).

5.8. Within-Regime Zero-Shot Generalization and Quantization under Shift

The Sen1Floods11 result bounds the recommendations across *sensors*; it leaves open how the models behave on unseen imagery from the regime they were built for — close-range RGB flood photography. Table 7 therefore evaluates the FSSD-trained models, without any fine-tuning, on two within-regime external sets with verified zero image overlap against FSSD (Section 3.1): AIFloodSense (94 ground-level flood photographs) and the held-out FloodNet test split (424 UAV images, 215 of which contain no water). Each configuration is evaluated with all three folds' checkpoints; we report mean ± cross-fold standard deviation. Three results follow.

First, FP32 students transfer as well as the teacher. On AIFloodSense every FP32 model lands within 0.69–0.72 IoU with cross-fold standard deviations below 0.015; the best transfer is MobileViT-XXS (0.723 ± 0.006), and the teacher holds no advantage (0.706 ± 0.004). Combined KD transfers slightly *worse* than no-KD on all three architectures (−0.005 to −0.015 IoU), extending the negative distillation result of Sec-

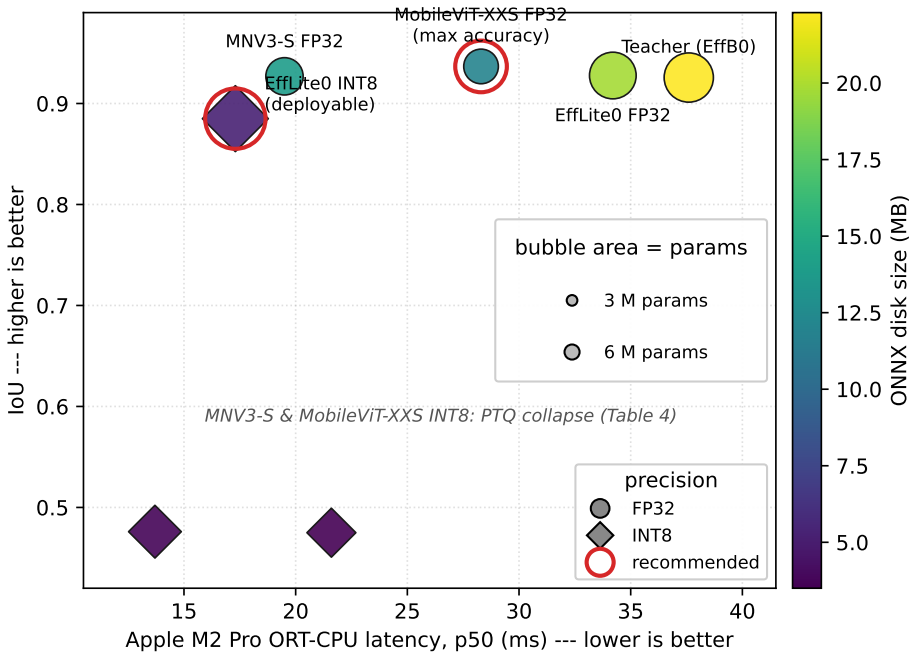


Figure 2: **Consolidated accuracy–latency–size–precision trade-off.** Each configuration is placed by Apple M2 Pro ORT-CPU p50 latency (x) and IoU (y); bubble area is proportional to parameter count and colour encodes ONNX disk size, so INT8 quantization moves a model left (faster) and toward a smaller, darker marker. Circles are FP32, diamonds INT8. The two red-ringed points are the recommended operating points — **EfficientNet-Lite0 INT8** (maximum throughput at acceptable accuracy) and **MobileViT-XXS FP32** (maximum accuracy); the two INT8 diamonds at IoU ≈ 0.48 are MobileNetV3-Small and MobileViT-XXS, whose static-QDQ INT8 collapses (Table 4). The teacher (yellow, 22 MB) offers no M2 latency advantage over the FP32 students. Browser WASM latencies are in Table 5; the four single-axis Pareto panels of the earlier draft remain in the repository as `figures/F2_pareto.pdf`.

tion 5.2 beyond the training distribution. On FloodNet's flood-containing images, all FP32 models cluster at 0.47–0.51 flood-IoU — a uniform ~0.42-point drop from in-domain performance that reflects the viewpoint and resolution gap of high-altitude UAV imagery rather than any architecture-specific failure.

Second, FSSD-trained models cannot say "no flood". Every FP32 model, teacher included, predicts water on 72–100% of the 215 water-free FloodNet scenes (Table 7, FAR column; MobileViT false-alarms on all of them in every fold). The cause is structural: FSSD contains flood imagery only, so the decision "this scene contains no water" is never supervised. For the monitoring use-cases that motivate edge deployment — continuous observation of scenes that are dry most of the time — this failure mode is disqualifying independently of segmentation quality, it is invisible to every FSSD-internal metric in Tables 2–4, and it compounds the operational error asymmetry of Section 6.5 in the costly direction. Retraining on negative-image-bearing data (Section 3.1) is the direct remedy and is left to future work.

Third, INT8 degradation grows with distribution shift. In-domain, EfficientNet-Lite0-INT8 loses 0.043 IoU (Table 4); zero-shot it loses 0.053 on AIFloodSense (0.702 → 0.649) and 0.054 flood-IoU on FloodNet (0.507 → 0.453), while its false-alarm rate jumps from 0.727 to 0.932 — the quantized model fails the water-free scenes that its FP32 parent still partially rejects. The fragile architectures collapse outright: MobileNetV3-Small-INT8 drops to 0.192 ± 0.164 IoU on AIFloodSense and 0.077 ± 0.073 flood-IoU on FloodNet — its seemingly moderate overall FloodNet score (0.355) comes from predicting almost nothing, which is rewarded on the 215 water-free images — and MobileViT-XXS-INT8 loses more than half its FP32 transfer (0.723 → 0.337). Per-tensor PTQ calibrated on 80 in-domain images therefore not only fails architecture-dependently (Section 5.3) but degrades further under the distribution shift that defines real deployments, most visibly in false-alarm behaviour; we flag this as a practitioner trap that in-domain PTQ validation does not reveal, and note that it also illustrates why mixed test sets must be reported with split metrics rather than a single macro IoU.

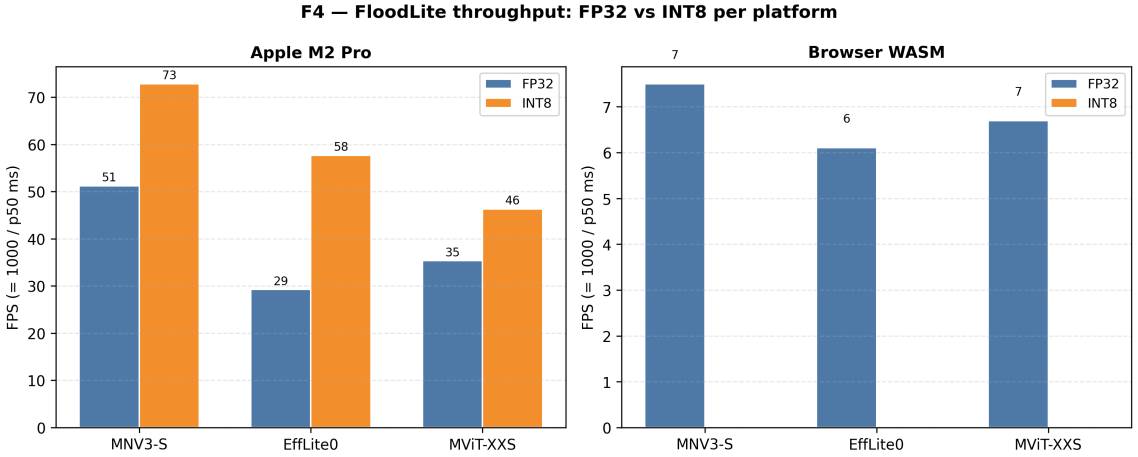


Figure 3: **Per-platform throughput, FP32 vs. INT8.** Frames-per-second on M2 Pro (left) and browser WASM SIMD (right); INT8 entries are unsupported on ORT-Web 1.17.

Table 7: **Zero-shot generalization of FSSD-trained models** (no fine-tuning; mean ± std across the three folds' checkpoints) on AIFloodSense (94 images, all flood-containing) and the held-out FloodNet test split (424 images, 215 of them water-free). Per-image macro metrics (Section 4.3): IoU over all images, flood-only IoU over flood-containing images, and false-alarm rate (FAR) over water-free images. FSSD contains no water-free images, and every FP32 model false-alarms on 72–100% of water-free FloodNet scenes. INT8 degradation exceeds its in-domain counterpart (cf. Table 4): MobileNetV3-Small-INT8's seemingly moderate overall FloodNet IoU is an artifact of near-constant background prediction (flood-only IoU 0.08).

		AIFloodSense	FloodNet test		
Model	**Prec.**	**IoU**	**IoU (all)**	**IoU (flood)**	**FAR**
Teacher (UNet+EffB0)	FP32	0.706 ± 0.004	0.288 ± 0.014	0.509 ± 0.010	0.927 ± 0.018
MobileNetV2 baseline	FP32	0.690 ± 0.014	0.378 ± 0.015	0.475 ± 0.020	0.716 ± 0.040
MobileNetV3-Small	FP32	0.698 ± 0.008	0.285 ± 0.005	0.506 ± 0.012	0.929 ± 0.021
EfficientNet-Lite0	FP32	0.702 ± 0.006	0.388 ± 0.025	0.507 ± 0.021	0.727 ± 0.032
MobileViT-XXS	FP32	0.723 ± 0.006	0.253 ± 0.004	0.512 ± 0.007	1.000 ± 0.000
MobileNetV3-Small	INT8	0.192 ± 0.164	0.355 ± 0.140	0.077 ± 0.073	0.375 ± 0.346
EfficientNet-Lite0	INT8	0.649 ± 0.028	0.258 ± 0.011	0.453 ± 0.032	0.932 ± 0.012
MobileViT-XXS	INT8	0.337 ± 0.078	0.092 ± 0.019	0.172 ± 0.049	0.986 ± 0.011

6. Discussion

6.1. *The Negative-KD Finding and What It Means for the Field*

The central empirical claim of this paper is that knowledge distillation does not help on FSSD-class flood segmentation when the student is an ImageNet-pretrained lightweight UNet. We frame this as a *negative result* rather than a *null result* because the comparison is not "we did not find a positive effect" but rather "the best student (MobileViT-XXS, no-KD) *exceeds* the teacher (UNet+EfficientNet-B0)". Distillation on this benchmark is at best redundant and at worst counterproductive: combined KD on MobileViT-XXS produced −0.005 IoU vs. the same architecture trained without it.

We interpret this mechanistically. The classical motivation for KD — Hinton's *dark knowledge* argument [23] and subsequent feature-map alignment work [25, 26, 27, 28] — assumes that the student has insufficient capacity or pretraining to recover the teacher's behaviour from labelled data alone, so the teacher's soft probabilities and intermediate features provide a richer supervision signal than the binary mask. On FSSD, that assumption is violated in two directions: (i) the benchmark has only 663 images and saturates at IoU ≈ 0.93 — the labelled signal is already nearly sufficient — and (ii) the student's ImageNet pretraining already supplies the visual priors (edge detectors, texture filters, semantic-region features) that a teacher trained on the same FSSD would have been transferring. The teacher's "dark know-

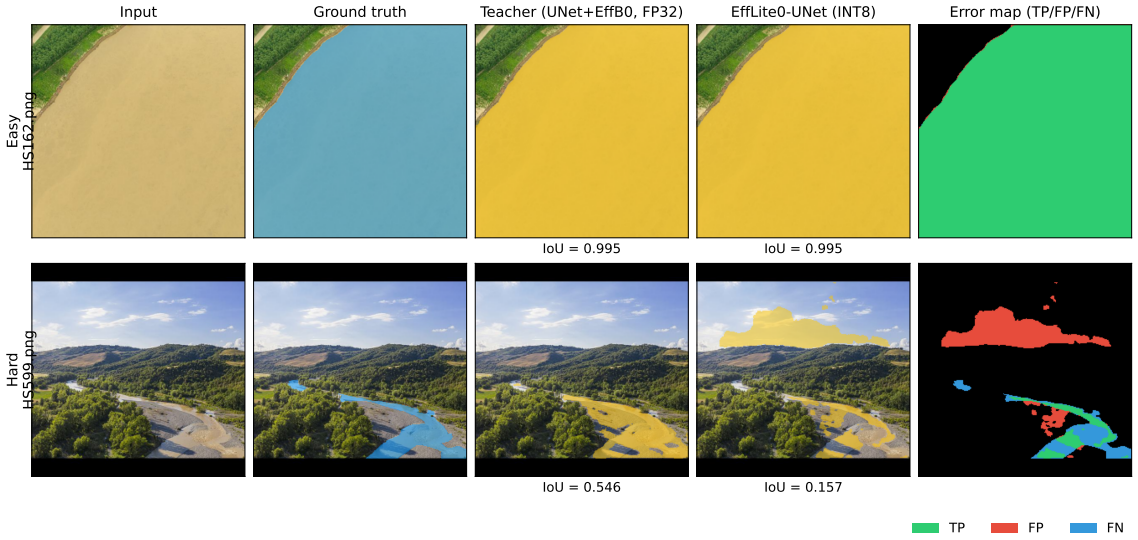


Figure 4: **Qualitative comparison on FSSD fold-0 validation.** Two rows × five columns: input, ground truth, teacher (UNet+EfficientNet-B0, FP32) prediction, EfficientNet-Lite0-UNet INT8 prediction, and error map (TP green, FP red, FN blue). **Easy row** (top): the highest-IoU validation image under the deployable INT8 student (FSS1604, IoU = 0.995 for both teacher and student). **Hard row** (bottom): the lowest-IoU image with ≥ 2% flood-pixel coverage (FSS904, teacher IoU = 0.546; student IoU = 0.157). The hard row shows the two persistent failure modes discussed in Section 6.4: false-positive activations on bright reflective non-water surfaces (red regions), and false-negative under-segmentation along vegetation-occluded boundaries (blue regions).

ledge" is therefore largely redundant with the student's pretraining, and the response-based KL term reduces to a noise-injection regulariser. The feature-MSE term performs slightly better — it provides at least *some* signal beyond the labels — but the gain is well within fold variance and is not statistically significant at our reportable resolution.

This finding has two implications for the flood-segmentation literature:

1. **Benchmarks matter.** A KD recipe that wins on Cityscapes (19 classes, 5000 images, IoU ≈ 0.80–0.85) does not transfer to FSSD (2 classes, 663 images, IoU ≈ 0.93). Future KD work in this space should report headroom (teacher–best-pretrained-student gap) *before* claiming distillation effects.
2. **Modern ImageNet pretraining is doing the work.** The 2026 wave of `timm`-pretrained lightweight backbones brings the no-KD baseline so high that compression strategies *other than* distillation — quantization, pruning, architecture choice — dominate the deployment-relevant trade-offs. We recommend that authors investing in flood-segmentation compression budget their effort accordingly.

6.2. *Per-Platform Deployment Recommendations*

Based on the joint segmentation × latency × stability evidence (Sections 5.2–5.4), we make the following deployment recommendations:

- **Field-laptop / tablet inference (Apple Silicon class):** EfficientNet-Lite0 UNet INT8. 17 ms p50 (58 FPS) at IoU 0.885, 3.83× size reduction. Stable across folds (±0.015). This is the configuration we ship in our public release.
- **Browser citizen-science / web-portal inference:** MobileNetV3-Small UNet FP32 (133 ms p50; 7.5 FPS), or MobileViT-XXS UNet FP32 (149 ms p50; 6.7 FPS) for slightly higher accuracy. INT8 is not yet supported in ONNX Runtime Web 1.17 and is gated on ORT-Web 1.18+ shipping a QDQ INT8 Conv kernel.
- **Maximum-accuracy off-line inference:** MobileViT-XXS UNet FP32 (28 ms p50 on M2 Pro CPU; IoU 0.937). Recommended for human-in-the-loop annotation pipelines.

We still do not recommend the teacher (UNet+EffB0) for deployment, but for a subtler reason than raw FP32 speed. On a common ONNX Runtime CPU backend the teacher runs at 37.6 ms p50 (26.6 FPS), only 1.1–1.9× slower than the FP32 students, so the case against it rests on *model size and quantization headroom* rather than FP32 latency: the recommended EfficientNet-Lite0

INT8 is 2.2× faster (17.3 vs 37.6 ms) and roughly 4× smaller (5.2 vs 22.3 MB) while recovering 95% of the teacher's IoU, and the teacher does not INT8-quantize within our budget.

Table 8 consolidates these recommendations into a decision matrix, pairing each operational scenario with the configuration we recommend and the metric that decides it.

All of these recommendations carry the zero-shot caveats of Sections 5.7 and 5.8: the artifacts are validated for in-domain, close-range RGB; FSSD-trained models false-alarm on the majority of water-free scenes and must be retrained on negative-image-bearing data (Section 3.1) before *unattended* monitoring use; and INT8 artifacts in particular should be re-validated on in-deployment-domain imagery, since their degradation worsens under distribution shift in a way that in-domain calibration metrics do not predict.

6.3. Architecture-Dependent PTQ Stability is a Practitioner Trap

The 0.45-IoU PTQ collapse on MobileNetV3-Small and MobileViT-XXS would not be predicted from the classification-quantization literature, which typically reports ∼ 1 IoU point PTQ tax on ResNet/MobileNet/EfficientNet families on ImageNet [39, 38]. The flood-segmentation regime differs in two respects: (i) the output is a per-pixel binary mask, so any quantization error that biases the post-sigmoid threshold materially shifts the segmentation; and (ii) flood images contain a wide dynamic range of bright reflective surfaces that drive calibration statistics in HardSwish-based backbones. We have observed that the same Percentile-99.999% recipe that fully recovers EfficientNet-Lite0 drives MobileNetV3-Small to sharply different per-fold INT8 outcomes: across folds 0–2 its INT8 IoU is 0.772, 0.0001 (near-total collapse to the background class), and 0.657 — one catastrophic failure and two partial degradations, with no fold recovering to its FP32 level (≈ 0.93). The large cross-fold standard deviation (±0.34) reflects this instability rather than a smooth degradation, which is why the per-fold values, not the mean, are the informative summary here.

The practical guidance: when assembling a flood-segmentation deployment artifact under a static-QDQ-INT8 budget, prefer backbones with bounded activation ranges (ReLU6 is the safest, ReLU is fine, HardSwish is risky, GELU and softmax-attention are *not* PTQ-friendly without QAT). Without this guidance, a practitioner running the standard ONNX Runtime quantization tutorial on MobileNetV3-Small would conclude — incorrectly — that the model "doesn't quantize" rather than that the recipe is mismatched to the activation family.

6.4. Failure Modes (Qualitative)

Figure 4 shows that both teacher and EfficientNet-Lite0-INT8 student share two persistent failure modes:

- **Bright-reflective-surface confusion:** wet asphalt, glare-on-roof, and large reflective puddles are misclassified as flood. This is a labelling-modality issue, not a model-capacity issue — RGB alone cannot disambiguate water from wet non-water surfaces.
- **Vegetation-occluded edges:** when flooded water is partially overhung by trees or tall grass, both models under-segment the flood boundary. The 5-pixel margin at the boundary contributes disproportionately to the residual IoU gap.

Both failure modes argue for SAR or multispectral fusion rather than for a different RGB segmentation recipe. We discuss this in Section 7.

Table 9 organises the failure modes observed across the whole study — the two RGB-modality failures above, the cross-sensor distribution shift of Section 5.7, and the INT8 PTQ instability of Section 5.3 — into a single taxonomy, noting for each its mechanism, error direction, the models affected, and the mitigation it implies. The categories are qualitative: we report the error direction (which matters for the operational cost asymmetry of Section 6.5) but do not claim per-class frequencies.

6.5. Operational Error Asymmetry

The metrics we report — IoU and F1 — weight false positives and false negatives equally, but in flood response the two errors carry very different operational costs. A false negative (unmapped flood) can translate into under-warning and under-allocation of rescue resources, whereas a false positive (over-flagged flood) mainly costs excess caution; for most disaster-response decisions the false negative is the more expensive error. We report precision and recall separately in Table 2 precisely so this trade-off is visible rather than hidden inside a single symmetric score: the FP32 teacher is close to balanced (precision 0.962, recall 0.960), and the two qualitative failure modes above pull in opposite directions — bright-surface confusion inflates false positives, vegetation occlusion inflates false negatives.

INT8 conversion further perturbs this balance, and not in a stable direction: the deployable EfficientNet-Lite0 student's per-fold (precision, recall) after quantization

Table 8: **Deployment decision matrix.** Recommended configuration per operational scenario, with the deciding metrics. All values are drawn from Tables 1, 2, 4, 5, and 6; this table re-presents the Section 6.2 recommendations as an at-a-glance practitioner reference and introduces no new experiments. Latency is p50 on the platform relevant to each scenario (Apple M2 Pro ORT-CPU, or browser WASM SIMD).

Deployment scenario	Recommended	IoU	Latency (p50)	Size (MB)	Rationale / caveat
Field laptop / tablet (Apple-Silicon CPU)	**EffLite0-UNet INT8**	0.885	17 ms (M2)	5.2	Fastest deployable operating point; the only student that INT8-quantizes stably (±0.015). Shipped in our release.
Browser / citizen-science web portal	MNV3-S-UNet FP32	0.927	133 ms (WASM)	13.8	Fastest under single-threaded WASM SIMD; MobileViT-XXS FP32 (149 ms, IoU 0.937) if accuracy matters more. No INT8 in ORT-Web 1.17.
Maximum-accuracy off-line / human-in-the-loop	MobileViT-XXS-UNet FP32	0.937	28 ms (M2)	12.0	Highest IoU of any configuration; FP32 only (its INT8 collapses, Table 4).
Cross-sensor / satellite optical	*retrain / domain-adapt*	0.16[†]	—	—	No FSSD-trained model transfers zero-shot (Table 6); requires domain adaptation or in-sensor training data.

[†]Zero-shot Sen1Floods11 IoU; all models ≈ 0.16 (Section 5.7).

are (0.965, 0.934), (0.917, 0.963), and (0.897, 0.963) — recall-favouring (safer, over-segmenting) in two folds but precision-favouring (missing flood) in the third. We therefore make two recommendations for operational use. First, tune the post-sigmoid decision threshold toward recall — accepting more false positives to reduce missed flood — rather than leaving it at the default 0.5, matching the asymmetric cost structure. Second, re-calibrate that threshold *after* INT8 conversion, since quantization shifts the operating point by an amount and in a direction our results show is not predictable from the FP32 model alone. A full cost-weighted evaluation (e.g. an F_β score with $\beta > 1$, or a region-level warning-recall metric) is left to future work: the per-image cost model is application- and jurisdiction-specific and cannot be fixed once for all deployments.

7. Limitations

We acknowledge the following limitations.

Single-dataset training and cross-sensor transfer.. All training uses FSSD (663 RGB images, none of them water-free). We evaluate cross-domain transfer on 250 RGB chips from Sen1Floods11 (Section 5.7): every FSSD-trained model collapses to IoU ≈ 0.16 on this Sentinel-2 optical benchmark, so the recommended configurations do *not* transfer zero-shot across sensors and viewpoints. Within the close-range RGB regime, the overlap-verified zero-shot evaluation (Section 5.8) quantifies what single-dataset training costs — a ∼0.42-point flood-IoU drop on high-altitude UAV imagery and 72–100% false-alarm rates on water-free scenes — without removing the limitation: the recommended configurations should be retrained on negative-image-bearing data (Section 3.1) before unattended monitoring use. Cross-sensor deployment (e.g. to satellite optical or SAR) would require domain adaptation or in-sensor retraining, which we leave to future work. Because training draws on a single dataset, we likewise cannot claim broad geographic robustness even within the close-range-RGB regime.

RGB-only modality.. Operational disaster-response platforms increasingly fuse SAR (cloud-penetrating, all-weather) with optical imagery; our work is restricted to the RGB regime FSSD was constructed for. The two persistent failure modes in Section 6.4 (wet-asphalt confusion, vegetation occlusion) are intrinsic to the RGB modality and cannot be solved by a better RGB model.

Three folds at the fold level.. The 3-fold protocol gives a minimum reportable two-sided Wilcoxon p of 0.25 *at the fold level*, so we do not rely on fold-level hypothesis tests. Significance for the KD contrasts is instead established by the per-image paired bootstrap over the pooled held-out validation set ($n = 399$ images; Sections 4.4, 5.2), which has adequate power on the FSSD

Table 9: **Error taxonomy for FSSD flood segmentation.** Four failure classes observed across this study, their mechanism, the direction of the resulting error (operationally, false negatives are the costlier direction, Section 6.5), the models affected, and the mitigation each implies. Categories are qualitative, synthesised from Sections 5.3, 5.7, and 6.4; we do not claim per-class frequencies.

Failure class	**Mechanism**	**Error type**	**Affected models**	**Mitigation**
Reflective-surface confusion	RGB cannot disambiguate water from wet non-water surfaces (wet asphalt, roof glare, standing puddles)	False positive	Teacher and all students	SAR / multispectral fusion (outside the RGB scope)
Vegetation-occluded boundary	Flood edges overhung by trees or tall grass are under-segmented	False negative	Teacher and all students	Modality limitation; boundary-aware loss at best partial
Cross-sensor distribution shift	Close-range RGB → Sentinel-2 satellite optical domain gap	Whole-image collapse (IoU ≈ 0.16)	All (uniformly)	Domain adaptation or in-sensor retraining
INT8 PTQ instability	Unbounded HardSwish tail / attention dynamic range exceed static-QDQ calibration	Model-level collapse (ΔIoU ≈ −0.45, high cross-fold variance)	MobileNetV3-Small, MobileViT-XXS	QAT or a ReLU6-bounded backbone (EffLite0 unaffected)

test distribution. Two caveats remain. First, the bootstrap quantifies uncertainty over *images*, not over independent dataset draws, and the three fold training sets overlap by construction, so it does not license population-level generalisation claims — a larger fold count or the independent-dataset evaluation flagged above would be required for that. Second, the PTQ collapses are reported as effect-size magnitudes (e.g., the −0.45 IoU drop on MobileNetV3-S INT8 is unambiguous) rather than by hypothesis test, given their trimodal cross-fold behaviour (Table 4).

Two deployment platforms (M2 Pro + browser WASM).. We do not measure Raspberry Pi 4, AWS Graviton (ARM CPU), Android (Termux + TFLite), or any GPU edge target (Jetson Nano / Orin). Adding these is largely an artifact-conversion exercise — the ONNX hub artifact remains the same — but each adds a non-trivial setup and tuning cost we did not budget for. Pi 4 and Graviton numbers are the most directly comparable extensions and are scheduled for a `v1.1` supplementary release.

Post-training quantization only.. We do not evaluate quantization-aware training (QAT). The 0.45-IoU PTQ collapse on MobileNetV3-Small and MobileViT-XXS is a candidate target for QAT — both architectures are reportedly QAT-recoverable in the classification literature — but QAT is an order-of-magnitude larger training investment than the experiments reported here and is left to future work.

No energy / power measurements.. We do not report Joules-per-inference. Energy is the metric most directly relevant to battery-operated deployments (drones, field tablets) and we acknowledge its absence; the equipment required (USB-meter, INA219 / current-clamp logger) was not available during this study.

Saturated benchmark.. FSSD saturates at IoU ≈ 0.93 for the entire UNet + ImageNet-pretrained backbone family. The negative-KD finding is therefore strictly a claim about saturated-benchmark behaviour. We do *not* claim that KD never helps on flood segmentation in general — only that, on this specific benchmark, it does not. A non-saturated benchmark with limited labels, mixed imaging modalities, or larger viewpoint diversity might recover the KD signal.

8. Conclusions

We presented FloodLite, the first systematic CPU-and-browser edge-deployment benchmark for lightweight flood segmentation. Across three lightweight UNet students, four knowledge-distillation configurations, three cross-validation folds, two execution providers, and FP32 + INT8 precisions, we have established four results that we believe will be useful to the operational disaster-response community:

1. ImageNet-pretrained lightweight UNets match or exceed a UNet+EfficientNet-B0 teacher

on FSSD *without* distillation, and combined response+feature KD does not improve over no-KD on any of the three architectures.

2. Post-training INT8 quantization stability is sharply architecture-dependent, with bounded-activation backbones (ReLU6) quantizing cleanly while HardSwish-based and attention-based backbones require quantization-aware training.
3. The recommended deployable configuration, EfficientNet-Lite0-UNet INT8 served by ONNX Runtime CPU, runs at 58 FPS on a 2023 laptop CPU at 3.83× size reduction with IoU 0.885 (95% of the teacher's accuracy) — a 2.2× speedup over the teacher on the same ONNX Runtime CPU backend; in the browser, where INT8 is unsupported by ONNX Runtime Web 1.17, an FP32 student runs at 6–7.5 FPS in a single-threaded WebAssembly sandbox (MobileNetV3-Small fastest).
4. Zero-shot evaluation bounds the deployment claims twice over. Across sensors, every FSSD-trained model collapses on Sentinel-2 optical imagery (IoU $\approx$ 0.16). Within the close-range RGB regime, on overlap-verified external data, FP32 students transfer as well as the teacher (0.69–0.72 IoU), but flood-only training data produces models that false-alarm on 72–100% of water-free scenes, and INT8 degradation grows under domain shift, up to outright collapse — so in-domain PTQ validation alone cannot qualify a quantized artifact for deployment.

The same `.onnx` artifact serves both targets, which we offer as a practical pattern for flood-segmentation deployment on heterogeneous edge hardware.

We release our trained checkpoints, the ONNX FP32 and INT8 exports, all benchmarking harnesses, and a Zenodo-archived snapshot of the code repository under permissive licences. Future work includes retraining on negative-image-bearing combined data (Section 3.1) to remove the false-alarm failure mode of Section 5.8, domain adaptation for cross-sensor transfer — our zero-shot Sen1Floods11 evaluation collapses to IoU $\approx$ 0.16 (Section 5.7), so the recommendations hold only for in-domain close-range RGB — ARM CPU benchmarks on Raspberry Pi 4 and AWS Graviton, energy measurements, cost-weighted (recall-favouring) evaluation matched to the operational error asymmetry (Section 6.5), and quantization-aware-training of the HardSwish and attention-based backbones whose post-training quantization stability we have characterised as a current practitioner trap.

Author Contributions

Conceptualisation, T.N.C. and M.I.K.; methodology, M.I.K.; software, M.I.K.; validation, M.I.K.; investigation, T.N.C., M.I.K. and M.I.H.; data curation, M.I.K.; writing — original draft preparation, M.I.K.; writing — review and editing, all authors; visualisation, M.I.K.; project administration, M.I.K. All authors have read and agreed to the submitted version of the manuscript.

Funding

This research received no external funding.

Data Availability Statement

The Flood Semantic Segmentation Dataset (FSSD) is publicly available at `https://www.kaggle.com/datasets/lihuayang111265/flood-semantic-segmentation-dataset`. All trained models, training code, and deployment scripts produced for this study are available at `https://github.com/m-ibrahim-khalil/flood-segmentation` (MIT licence; Zenodo DOI to be minted at submission). The Sen1Floods11 dataset, used in preliminary out-of-distribution preparation, is available at `https://github.com/cloudtostreet/Sen1Floods11` (CC-BY 4.0).

Conflicts of Interest

The authors declare no conflict of interest.

Acknowledgements

The authors thank the Kaggle community for the public FSSD dataset and the ONNX Runtime team for the static QDQ quantization API used in this work.

References

[1] W. Du, G. J. FitzGerald, M. Clark, X.-Y. Hou, Health impacts of floods, Prehospital and Disaster Medicine 25 (3) (2010) 265–272. `doi:10.1017/S1049023X00008141`.

[2] M. Hemmati, B. R. Ellingwood, H. N. Mahmoud, The role of urban growth in resilience of communities under flood risk, Earth's Future 8 (3) (2020) e2019EF001382. `doi:10.1029/2019EF001382`.

[3] V. Kumar, K. V. Sharma, T. Caloiero, D. J. Mehta, K. Singh, Comprehensive overview of flood modeling approaches: A review of recent advances, Hydrology 10 (7) (2023) 141. doi:10.3390/hydrology10070141.

[4] R. Bentivoglio, E. Isufi, S. N. Jonkman, R. Taormina, Deep learning methods for flood mapping: A review of existing applications and future research directions, Hydrology and Earth System Sciences Discussions (2022) 1–50doi:10.5194/hess-2022-83.

[5] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241. doi:10.1007/978-3-319-24574-4_28.

[6] V. Badrinarayanan, A. Kendall, R. Cipolla, Segnet: A deep convolutional encoder-decoder architecture for image segmentation, IEEE Transactions on Pattern Analysis and Machine Intelligence 39 (12) (2017) 2481–2495. doi:10.1109/TPAMI.2016.2644615.

[7] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam, Encoder-decoder with atrous separable convolution for semantic image segmentation, in: European Conference on Computer Vision (ECCV), 2018, pp. 801–818. doi:10.1007/978-3-030-01234-2_49.

[8] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.

[9] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. doi:10.1109/CVPR.2016.90.

[10] Ş. Karcı, E. Özceylan Aslan, K. Yıldız, Ö. Demir, Deep learning based flood segmentation: Evaluating efficientnet and resnet for unet, segnet and deeplabv3+, Natural Hazards 122 (2026) 263. doi:10.1007/s11069-026-07972-7.

[11] F. Pech-May, J. V. Sanchez-Hernández, L. A. López-Gómez, et al., Flooded areas detection through sar images and u-net deep learning model, Computación y Sistemas 27 (2023) 449–458.

[12] I. Chamatidis, D. Istrati, N. D. Lagaros, Vision transformer for flood detection using satellite images from sentinel-1 and sentinel-2, Water 16 (12) (2024) 1670. doi:10.3390/w16121670.

[13] D. Hernández, J. M. Cecilia, J.-C. Cano, C. T. Calafate, Flood detection using real-time image segmentation from unmanned aerial vehicles on edge-computing platform, Remote Sensing 14 (1) (2022) 223. doi:10.3390/rs14010223.

[14] H. S. Munawar, A. W. A. Hammad, S. T. Waller, A review on flood management technologies related to image processing and machine learning, Automation in Construction 132 (2021) 103916. doi:10.1016/j.autcon.2021.103916.

[15] F. Schmalstieg, K. Mueller, W. Samek, Cross-architecture foundation-model distillation for edge flood segmentation, arXiv preprint arXiv:2609.20441, submitted to IEEE JSTARS (2026).

[16] Y. Lin, T. Zhang, P. Sun, Z. Li, S. Zhou, FQ-ViT: Post-training quantization for fully quantized vision transformer, in: International Joint Conference on Artificial Intelligence (IJCAI), 2022, pp. 1173–1179.

[17] B. Bahrami, H. Arbabkhah, Enhanced flood detection through precise water segmentation using advanced deep learning models, Journal of Civil Engineering Researchers 6 (2024) 1–8.

[18] V. Patil, Y. Khadke, A. Joshi, et al., Flood mapping and damage assessment using ensemble model approach, Sensing and Imaging 25 (2024) 15. doi:10.1007/s11220-024-00467-4.

[19] B. Ghosh, S. Garg, M. Motagh, et al., Automatic flood detection from sentinel-1 data using a nested unet model and a nasa benchmark dataset, PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science 92 (2024) 1–18. doi:10.1007/s41064-024-00275-3.

[20] G. Mateo-Garcia, J. Veitch-Michaelis, L. Smith, S. V. Oprea, G. Schumann, Y. Gal, A. G. Baydin, D. Backes, Towards global flood mapping onboard low cost satellites with machine learning, Scientific Reports 11 (2021) 7249. doi:10.1038/s41598-021-86650-z.

[21] Y. Nie, Z. Shi, J. Li, H. Ge, Gated lightweight CNN–Transformer fusion for real-time flood segmentation on satellite internet terminals under triple-disruption emergency conditions, Remote Sensing 18 (9) (2026) 1418. doi:10.3390/rs18091418.

[22] R. Bhattacharjya, Y. Jung, M. Kim, S.-Y. Wu, E. Bozorgzadeh, N. Venkatasubramanian, N. Dutt, FloodReasonBench: Benchmarking VLM reasoning segmentation for embodied flood response at the edge, arXiv preprint arXiv:2608.15410 (2026).

[23] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint (2015). arXiv:1503.02531.

[24] J. Gou, B. Yu, S. J. Maybank, D. Tao, Knowledge distillation: A survey, International Journal of Computer Vision 129 (2021) 1789–1819. doi:10.1007/s11263-021-01453-z.

[25] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, Y. Bengio, Fitnets: Hints for thin deep nets, in: International Conference on Learning Representations (ICLR), 2015.

[26] Y. Liu, K. Chen, C. Liu, Z. Qin, Z. Luo, J. Wang, Structured knowledge distillation for semantic segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 2604–2613. doi:10.1109/CVPR.2019.00271.

[27] Y. Wang, W. Zhou, T. Jiang, X. Bai, Y. Xu, Intra-class feature variation distillation for semantic segmentation, in: European Conference on Computer Vision (ECCV), 2020, pp. 346–362. doi:10.1007/978-3-030-58571-6_21.

[28] K. He, Y. Shi, X. Sun, R. Jiang, X. Lin, X. Cui, Channel-wise knowledge distillation for dense prediction, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 5311–5320. doi:10.1109/ICCV48922.2021.00526.

[29] J. H. Cho, B. Hariharan, On the efficacy of knowledge distillation, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 4794–4802.

[30] L. Beyer, X. Zhai, A. Royer, L. Markeeva, R. Anil, A. Kolesnikov, Knowledge distillation: A good teacher is patient and consistent, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 10925–10934. doi:10.1109/CVPR52688.2022.01065.

[31] S. Stanton, P. Izmailov, P. Kirichenko, A. A. Alemi, A. G. Wilson, Does knowledge distillation really work?, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 34, 2021, pp. 6906–6919.

[32] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, Q. Le, Searching for mobilenetv3, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 1314–1324. doi:10.1109/ICCV.2019.00140.

[33] S. Mehta, M. Rastegari, Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer, in: International Conference on Learning Representations (ICLR), 2022.

[34] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.

[35] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, N. Sang, Bisenet v2: Bilateral network with guided aggregation for real-time semantic segmentation, International Journal of Computer Vision 129 (2021) 3051–3068. doi:10.1007/s11263-021-01515-2.

[36] J. Xu, Z. Xiong, S. P. Bhattacharyya, Pidnet: A real-time semantic segmentation network inspired by pid controllers, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 19529–19539. doi:10.1109/CVPR52729.2023.01871.

[37] R. P. K. Poudel, S. Liwicki, R. Cipolla, Fast-scnn: Fast semantic segmentation network, in: British Machine Vision Conference (BMVC), 2019.

[38] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713. doi:10.1109/CVPR.2018.00286.

[39] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient inference: A whitepaper, arXiv preprint (2018). arXiv:1806.08342.

[40] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, K. Keutzer, A survey of quantization methods for efficient neural network inference, in: Low-Power Computer Vision, Chapman and Hall/CRC, 2022, arXiv:2103.13630.

[41] ONNX Runtime Developers, Onnx runtime v1.17 – static qdq quantization recipe, https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html, accessed 16 May 2026 (2025).

[42] Z. Li, J. Xiao, L. Yang, Q. Gu, RepQ-ViT: Scale reparameterization for post-training quantization of vision transformers, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 17227–17236.

[43] H. Qiu, I. Vavelidou, J. Li, E. Pergament, P. Warden, S. Chinchali, Z. Asgar, S. Katti, ML-EXray: Visibility into ML deployment on the edge, in: Proceedings of Machine Learning and Systems (MLSys), Vol. 4, 2022, pp. 337–351.

[44] M. M. Kuglitsch, I. Pelivan, S. Ceola, M. Menon, E. Xoplaki, Facilitating adoption of ai in natural disaster management through collaboration, Nature Communications 13 (2022) 1579. doi:10.1038/s41467-022-29285-6.

[45] E. Nemni, J. Bullock, S. Belabbes, L. Bromley, Fully convolutional neural network for rapid flood segmentation in synthetic aperture radar imagery, Remote Sensing 12 (16) (2020) 2532. doi:10.3390/rs12162532.

[46] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, et al., MLPerf Tiny benchmark, in: NeurIPS Datasets and Benchmarks Track, 2021, arXiv:2106.07597.

[47] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, S. Han, MCUNet: Tiny deep learning on IoT devices, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 11711–11722.

[48] D. Smilkov, N. Thorat, Y. Assogba, A. Yuan, N. Kreeger, P. Yu, K. Zhang, S. Cai, E. Nielsen, D. Soergel, et al., TensorFlow.js: Machine learning for the web and beyond, in: Proceedings of Machine Learning and Systems (MLSys), Vol. 1, 2019.

[49] lihuayang111265 (Kaggle contributor), Flood semantic segmentation dataset (fssd), https://www.kaggle.com/datasets/lihuayang111265/flood-semantic-segmentation-dataset, accessed 16 January 2026 (2024).

[50] M. Rahnemoonfar, T. Chowdhury, A. Sarkar, D. Varshney, M. Yari, R. R. Murphy, FloodNet: A high resolution aerial imagery dataset for post flood scene understanding, IEEE Access 9 (2021) 89644–89654. doi:10.1109/ACCESS.2021.3090981.

[51] G. Simantiris, K. Bacharidis, A. Papanikolaou, P. Giannakakis, C. Panagiotakis, AIFloodSense: A global aerial imagery dataset for semantic segmentation and understanding of flooded environments, Remote Sensing 18 (6) (2026) 938. doi:10.3390/rs18060938.

[52] F. Karim, Flood area segmentation, https://www.kaggle.com/datasets/faizalkarim/flood-area-segmentation, kaggle dataset v1, 290 images with self-annotated water masks; CC0 Public Domain. Accessed 2026-10-31 (2022).

[53] P. Iakubovskii, Segmentation models pytorch, https://github.com/qubvel/segmentation_models.pytorch (2019).

[54] R. Wightman, Pytorch image models, https://github.com/rwightman/pytorch-image-models (2019).

[55] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, A. A. Kalinin, Albumentations: Fast and flexible image augmentations, Information 11 (2) (2020) 125. doi:10.3390/info11020125.

[56] D. Bonafilia, B. Tellman, T. Anderson, E. Issenberg, Sen1Floods11: a georeferenced dataset to train and test deep learning flood algorithms for Sentinel-1, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2020, pp. 210–211.

Appendix A. Reproducibility

We provide the following artifacts for reproducibility.

Artifacts..

- **Code repository** (MIT licence) at `github.com/m-ibrahim-khalil/flood-segmentation`; the `v1.0-ijdrr` tag is the camera-ready commit. Zenodo DOI: `[to be minted at release]`.
- **Pre-trained checkpoints** (`*.pt` PyTorch state-dicts) under `code/runs/3fold/`: 14 configurations × 3 folds = 42 checkpoints, ∼ 700 MB total.
- **ONNX exports** (FP32 and INT8, per fold) for teacher and three students under `code/exports/`; ∼ 150 MB total.
- **Per-fold metrics** in `results.json` (FP32) and `quantized.json` (INT8) under `code/runs/3fold/`; re-derivable by `run_full_experiment.py`.
- **Per-platform latency** in `lat_m2.json` and `lat_wasm.json`; re-derivable by `bench_m2.py` and `bench.mjs` respectively.
- **Zero-shot evaluations** in `ood_fold{0,1,2}.json` (Sen1Floods11) and `zero_shot_fold{0,1,2}.json` (AIFloodSense + FloodNet test, per-image metrics incl. flood-only IoU and false-alarm rates); re-derivable by `eval_ood.py` and `eval_zero_shot.py`. The FSSD↔combined duplicate-check report is `fssd_overlap_report.json`, re-derivable by `check_fssd_overlap.py`.

Environment..

- Random seed: 42, fixed in the `KFold` constructor (data splits) and the notebook preamble; training-time stochasticity (weight init, augmentation order) is not pinned and is reflected in cross-fold standard deviations.
- Software: Python 3.11; PyTorch 2.8 with CUDA 12.1 (Kaggle T4) or Apple-Silicon MPS (M2); `segmentation-models-pytorch` ≥ 0.4; `timm` 1.0; `onnxruntime` 1.22.1 (M2 CPU; INT8 QDQ artifacts require ≥ 1.18); `onnxruntime-web` 1.17.3 (browser).
- Hardware: training on a single Kaggle Tesla T4 (16 GB); quantization and CPU latency on an Apple M2 Pro (10-core CPU, 16 GB unified memory, macOS 25.2); WASM latency via Node.js 25.
- Estimated reproduction cost: ∼ 35 Kaggle-T4 hours + ∼ 2 M2-CPU hours; wall-clock ≈ 3 calendar days assuming no Kaggle queue.